

UNIT 2: MPCA

$$\text{Performance} = \frac{1}{\text{Exec time}}$$

Pipelining

ARM7TDMI: Fetch $\rightarrow$ Decode $\rightarrow$ Execute

ARM9TDMI: Fetch $\rightarrow$ Decode $\rightarrow$ Execute $\rightarrow$ Memory $\rightarrow$ Writeback
IF ID EX MEM WB

ARM10: IF $\rightarrow$ issue $\rightarrow$ ID $\rightarrow$ EX $\rightarrow$ MEM $\rightarrow$ WB

Performance;

$$\text{Exec time}_{\text{pipeline}} = k \times t_{\text{clock}} \times 1_{\text{init}} + (n-1)_{\text{init}} \times t_{\text{clock}} = [k + (n-1)] t_{\text{clk}}$$

$$\text{Exec time}_{\text{non-pipe}} = k \times n \times t_{\text{clk}} = k \times t_p$$

$$\text{Speedup} = \frac{\text{Exec-time}_{\text{non-pipe}}}{\text{Exec time}_{\text{pipe}}} = \frac{k n t_{\text{clk}}}{[k + (n-1)] t_{\text{clk}}}$$

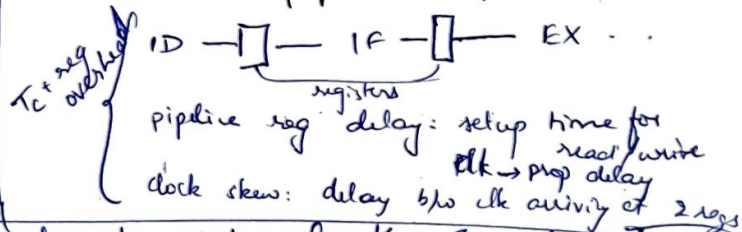
$$= \frac{k n t_{\text{clk}}}{(k-1+n) t_{\text{clk}}}$$

$k-1 \rightarrow$ negligible as n increases

$$\Rightarrow \frac{k n t_{\text{clk}}}{n t_{\text{clk}}} = k$$

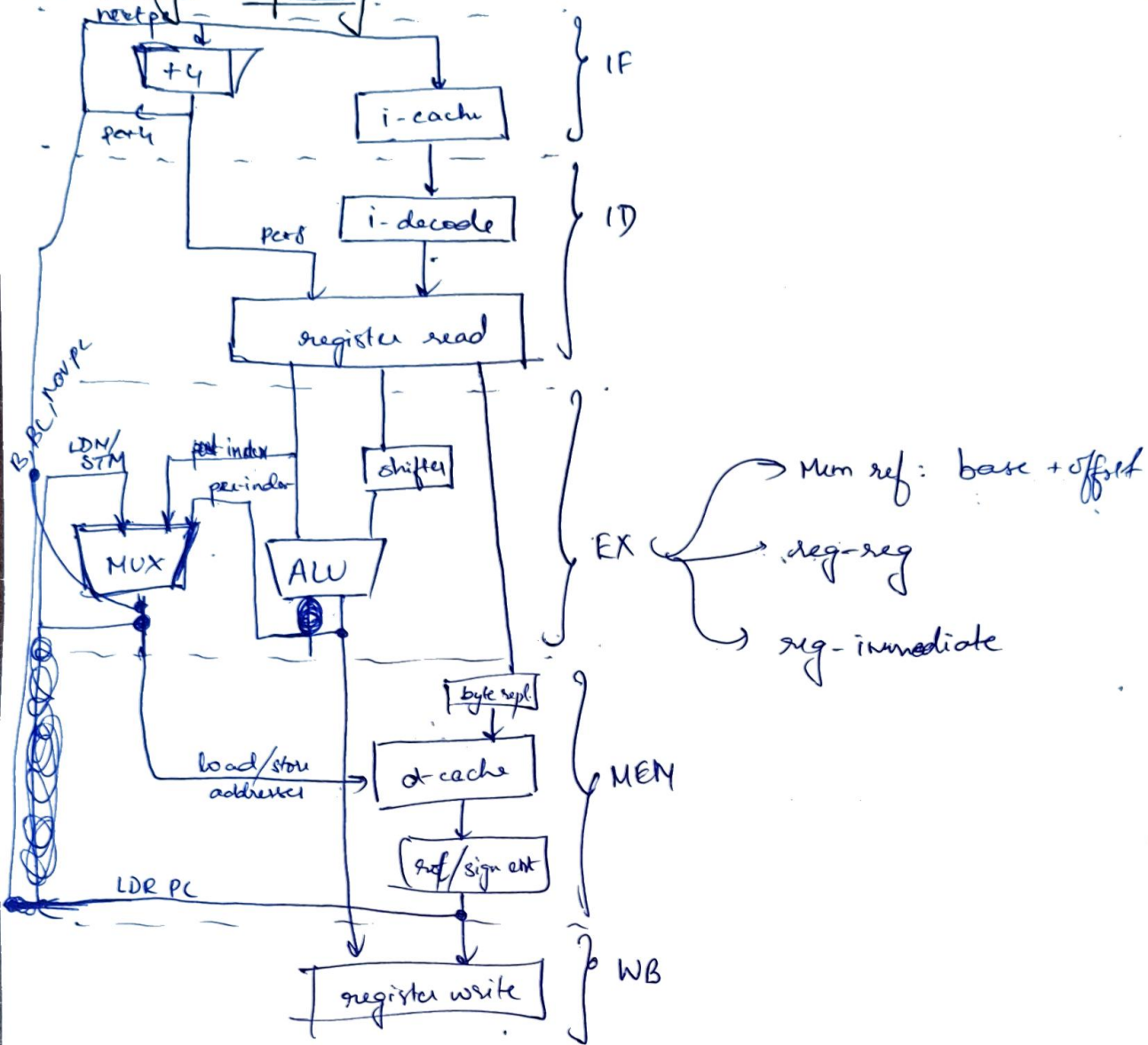
$$\text{Speedup (S)} = k$$

NOTE: pipeline reg overhead



NOTE: For pipelined, take length of each stage = length of longest stage

5 Stage Pipelining



HAZARDS

easy stupid way $\rightarrow$ stalls $\rightarrow$ insert "bubbles" to wait

Performance with stalls:

$$\text{Speedup (S)} = \frac{\text{CPI}_{\text{unpipe}} \times t_{\text{clk}_{\text{unpipe}}}}{\text{CPI}_{\text{pipe}} \times t_{\text{clk}_{\text{pipe}}}}$$

Assume overhead negligible:
 $t_{\text{chip}} = t_{\text{chip}}^{\text{simple}}$

$$S = \frac{CPI_{unpipe}}{CPI_{pipe}} = \frac{CPI_{unpipe}}{CPI_{ideal\ pipe} + Stall\ cycles}$$

$CPI_{ideal} \rightarrow 1$

$$CPT_{\text{unpipe}} = \text{Total stages of pipe} = \text{pipeline depth}$$

$$\Rightarrow S = \frac{\text{Pipeline depth} \times \text{ideal}}{1 + \text{stall cycles per instr.}}$$

②

Structural Hazards

Two instr $\rightarrow$ same hardware resource at the same time



unified memory (d+i)

only 1 reg file w/ port

IF vs MEM: both try to access memory (unified ~~both~~)

Soln: Add stalls $\times (3)$

Harvard memory architecture (split i-cache, d-cache)

WB vs WB: happens when instr doesn't need all stages

Soln: Stall
make it exec all stages

ID vs WB: both try to access register file (one port)

Soln: ~~stall~~ $\times$
partition: first half of cycle for reg ^{write} ~~read~~, second
for reg ~~read~~



Performance:

Say 30% are load/store instr.

$\Rightarrow$ 30% of instr MEM vs IF $\rightarrow$ 3 stall cycles

$$S = \frac{\text{pipeline depth} \times \text{ideal}}{\text{ideal} + \text{stall cycles per instr}} = \frac{5}{1 + (3 \times 0.3)} = \frac{5}{1.9} \approx 2.63 \text{ times faster instead of 5 times}$$

Data Hazards

RAW

reading before write
is done

add (R1), R2, R3
SUB R4, (R1), R3

WAR

writing before
read

add R4, (R1), R2
SUB (R1), R2, R3

WAW

second wr. before
first

ADD R1, . . .
SUB R1, . . .

doesn't happen
in pipeline



false dependency → remove

MUL R1, R2, R3
ADD R2, R4, R5

→ ADD (R6), R4, R5

Software soln → reorder instructions
insert ~~MOV~~ MOV R0, R0 (or) NOPs

Hardware soln → data forwarding → handles at EX, MEM
stalls

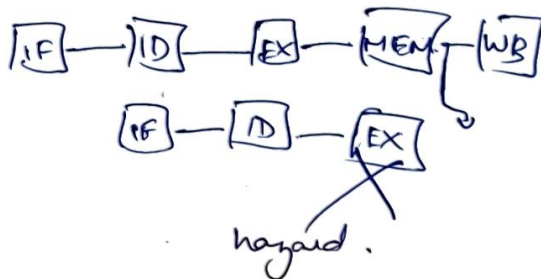
with nothing extra → 3 insts in b/w

with partitioning → 2

with partition. & data forwarding → 1

Even here you may need to stall after loads.

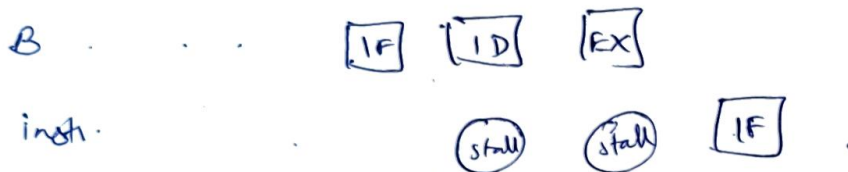
LDR R0, [R1, #60]
ADD R2, R0, R4



Control Hazard

When you have branches, next instr might be loaded before branch taken/not taken $\rightarrow$ useless, wrong order.

Solution ①: Stalls.

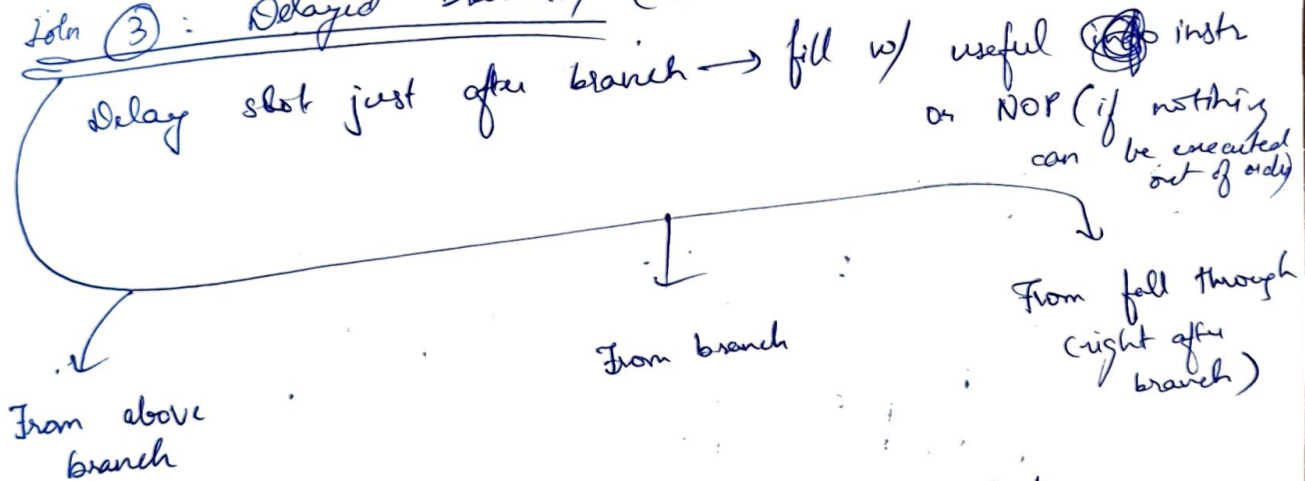


wasteful: 2 stalls for every branch

Soln ②: Compute Branch result in ID

reduces to 1 stall.

Soln ③: Delayed branching (no stall)



Best.

Requires instr copy

Requires the instr in question to be safe to execute regardless of taking/not taking branch.

Branch Prediction

1 bit dynamic (if wrong swap)

for ($i=0; i < n; i++$)

for ($i=0; i < n; i++$)
for ($j=0; j < m; j++$)

8xN hits

for ~~8~~ nxm iterations

$$\text{Accuracy} = \frac{8 \times n}{n \times m}$$

i	prediction	outcome
0	NT	T
1...n-1	T	T
n	T	NT

2 misses

i	j	pred	out
0	0	NT	T
0	1...m-1	T	T
0	m	T	NT
1	0	NT	T
...	...	...	...

2xN misses

$$8 + 2 = 10 = m$$

2 bit dynamic

i	j	pred	out
0	0	WNT	T
0	1...m-1	ST	T
0	m	ST	NT
1	0	WT	T
...	...	...	...

9xN hits

n+1 misses

$$9 + 1 = 10 = m$$

$$\text{Accuracy} = \frac{9 \times n}{n \times m}$$

